



High-Performance Computer Architectures Practical Course

Introduction to C++ and Unix Shell

Prof. Dr. Ivan Kisel

Robin Lakos

Akhil Mithran

Oddharak Tyagi

April 29, 2025

We would like to express our gratitude to Francesco Giacomini of INFN, Bologna, for generously permitting us to use his C++ course slides as inspiration for our introductory C++ lectures in this course.

Course Context: While this course is not primarily focused on teaching C++ programming, two introductory lectures are provided to cover essential concepts. These are designed to complement your existing programming knowledge and facilitate the application of course-specific concepts later on.

Lecture 1: Introduction to C++ and Unix shell

- Quick-start guide on SSH
- Short introduction of Unix shell
- Overview of C++ in the landscape of programming languages
- Basic principles to get started with C++ programming

Lecture 2: More advanced C++

- Deep dive into object-oriented programming in C++
- Introduction to meta-build tools essential for managing large projects

C++ is a powerful yet complex language. Effective use of C++ requires a deep understanding of its underlying mechanisms to prevent issues like undefined behavior, memory leaks or subtle bugs that might not be immediately apparent.

1 Connecting to University Computers from Home

2 Introduction to Unix Shell

3 Repetition: Computer Architectures and Programming

4 Introduction to C++

SSH (Secure Shell) is a protocol for securely accessing and managing a remote computer over an unsecured network.

It provides encrypted communication, allowing for safe command execution and file transfer between the client and server.

How to connect:

```
ssh [-AY] username@hostname [-p PORT]
```

Example:

```
ssh username@phoenix.rbi.cs.uni-frankfurt.de
```

Once the connection has been established, you can navigate via the terminal in the same way as if you were locally on the computer.

Difficulty Hard:

Terminal based text editors installed (without extensions): vim, nano

Difficulty Normal:

Connect to the remote machines via VS Code with the SSH extension and then use your familiar surroundings.

List of computers: <https://www.rbi.informatik.uni-frankfurt.de/RBI/en/about-us/pc-pool>

① Connecting to University Computers from Home

② Introduction to Unix Shell

③ Repetition: Computer Architectures and Programming

④ Introduction to C++

The Unix shell is a powerful command-line interface that allows users to interact directly with the operating system. It enables the execution of commands, manipulation of files, and navigation through directories.

Understanding basic shell commands is essential for efficient use of Unix or Linux-based systems.

Why learn Unix Shell commands?

- Direct and powerful control over the operating system
- Essential for programming, especially in environments where a graphical interface may not be available (e.g. HPC clusters)
- Foundation for advanced system administration, automation, and scripting tasks

Basic Unix Shell Commands

Some fundamental commands that are invaluable for daily tasks in the Unix Shell:

Command	Description
<code>pwd</code>	Print Working Directory. Shows the current directory you're in.
<code>cd <path></code>	Change Directory. Moves you to another directory. Use <code>cd ..</code> to go up one level.
<code>ls</code>	List. Shows files and directories in the current directory. Use <code>ls -l</code> for detailed listing.
<code>cp <source> <destination></code>	Copy. Copies files from the source to the destination.
<code>mv <source> <destination></code>	Move. Moves files from the source to the destination or renames a file.
<code>mkdir <directory></code>	Make Directory. Creates a new directory.
<code>rm <file></code>	Remove. Deletes a file. Use <code>rm -r <directory></code> to delete a directory and its contents.
<code>man <command></code>	Manual. Shows the manual page for a command, which includes how to use it and options.
<code>passwd</code>	Change password. However, on the machines on site, please use yppasswd to change it.

Practice these commands to become more familiar with the Unix shell environment. Remember, the `man` command is your friend for learning more about these and other commands.

Later during this lecture, we will compile and run our code using the terminal.

① Connecting to University Computers from Home

② Introduction to Unix Shell

③ Repetition: Computer Architectures and Programming

④ Introduction to C++

Modern computer architectures are inspired by the "Von Neumann" architecture (1945): Storing instructions and data together in a single read-write memory.

Central Processing Unit (CPU):

arithmetical-logical units, registers (incl. caches), control units

Random Access Memory (RAM):

volatile storage for data and instructions

Input-/Output:

persistent storage devices (HDDs, SSDs), graphics cards (GPUs), network adapters, keyboard, mouse, other external devices, ...

Depending on the specific architecture, there is a lot more to cover when it comes to performance: branch predictors, cache hierarchy, cache replacement strategies, cache mapping, clock speeds (CPU, RAM, GPU), memory (access) timings, ...

Most of these topics are taught in other (basic module) lectures (e.g. ARA, HL) and are probably already known. Keep them in mind when writing your assignments and analyzing your runtime.

Computer's Language vs. Programming Language (1)

Definitions:

A *computer* is a device that executes programs.

A *program* is a precise sequence of instructions enabling a computer to perform a specific task.

Language of a computer:

A computer only "understands" binary, as it is build from electronic parts that simply work based on present or not present voltage. Therefore, the signals send through a computer can only assume two states (0 and 1).

The exact language a computer "speaks" is based on the specific hardware (architecture). For example, an ARM-based processor speaks a different language than a x86_64-based processor.

The data used to perform specific tasks has to be in binary as well. The interpretation of the bits and bytes is additionally depending on the defined *data type*:

01100101 01110110 01101001 01101100 read as float (IEEE-754 32-Bit) is 7.272793e22

01100101 01110110 01101001 01101100 read as int (32-Bit) is 1702259052

01100101 01110110 01101001 01101100 read as ASCII/UTF-8 is "evil"

and also: byte order / endianness!

Programming language:

A (high-level) programming language is an artificial language developed for humans to give precisely defined instructions to a computer that are still close to human languages.

Since a computer only "speaks" binary, these instructions have to be translated into binary, which is usually done by a compiler or an interpreter.

The result of the compilation (the program in binary language) is usually called *executable*.

Why is an artificial (programming) language needed?

- Human languages have interpretative flexibility, while programming languages are designed with a specific syntax and semantics, meaning each instruction is strictly defined and performs a particular operation as dictated by the language's rules.

① Connecting to University Computers from Home

② Introduction to Unix Shell

③ Repetition: Computer Architectures and Programming

④ Introduction to C++

Online resources:

(1) C++ language and library reference:

- <https://cppreference.com/>
- somewhat formal, but precise, with examples

(2) C++ core guidelines:

- <https://isocpp.github.io/CppCoreGuidelines/>
- how to use C++ correctly

C++ "basics" covered in the lectures of course:

- Compilation
- Hello World
- References, Pointers
- Parameter-passing strategies: Pass-by-{ Reference, Pointer, Value }
- Undefined behavior (UB)
- Types of memory allocation (stack, heap)
- Templates (introduction)
- Object oriented programming (class, struct, template, overloading)
- Header- and source-files
- Linking
- Build tools and meta-build tools
- Optimization levels and compiler flags

How to C++:

- 1.) Write C++ code into a file (for now: file ending .cpp)
- 2.) Compile the code (translate code from C++ to binary executable)
- 3.) Run executable

To translate the code into an executable, a compiler is required.

On Linux-based systems, `g++` is usually installed per default, but the `clang++` compiler is another popular choice.

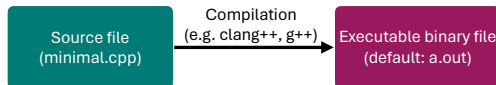
Attention: Although C++ compilers translate C++ into binaries, compilers sometimes have additional features that are not defined in the C++ standard. Usually these are quality of life features that can be used, but be aware that changing the compiler might lead to different error messages and also can make code unusable.

The Shortest C++ Program

Assume a C++ source file `minimal.cpp` with the following content:

```
int main() {}
```

Translation into executable:



Compile and run:

```
username@computername /home/username $ clang++ minimal.cpp -o minimal.out
```

```
username@computername /home/username $ ./minimal.out
```

```
username@computername /home/username $
```

and it does nothing, but successfully.

Hello World in C++

First create the source file `HelloWorld.cpp` for the simple "Hello World" console application.

```
#include <iostream>

int main(int argc, char** argv) {
    std::cout << "Hello world!" << std::endl;
}
```

Printing to console requires the Input/Output-Stream library. To import a standard library, `#include <...>` is used. For other (external) libraries, `#include "path/to/file"` is used, but `iostream` belongs to the standard.

The `std::` indicates the *namespace* (here standard library namespace) the functions (here `cout` and `endl`) are coming from.

Arrows `<<` and `>>` indicate a stream. The string "Hello world!" is streamed into the `std::cout`-function. This process returns `std::cout`, such that `std::endl` (end of line) is actually also streamed into `std::cout`.

Compile and run:

```
username@computername /home/username $ clang++ HelloWorld.cpp -o hello.out
username@computername /home/username $ ./hello.out
Hello world!
username@computername /home/username $
```

std::endl and Keywords using namespace

std::endl vs. \n:

The function-call result of `std::endl` looks similar to adding `"\n"` in the end of the string for a line break, but the behavior is different.

The `std::endl`-function also flushes the buffer to immediately print the result. In simple examples like above, both seem equal. However, sometimes, when `"\n"` is used and the program crashes before flushing the buffer, the results are not printed and it seems like the code of the print statement is not reached.

using namespace:

One can add `"using namespace NAMESPACE"` before using functions of a specific namespace.

This is not recommended!

For example, the statement `using namespace std;` in C++ is a directive that tells the compiler to bring all the names from the `std` namespace into the current namespace. This means you can use names (such as functions, objects, and types) defined in the `std` namespace without having to prefix them with `std::`.

Why not:

- All names of a namespace (especially `std`) are almost never required
- Everyone who includes this code is forced to also use the namespace
- Risk of naming conflicts with own or other included external codes
- Origin of a function is immediately known to a reader when prefix is used (readability)

There are several *primitive* or *fundamental data types* in C++, such as:

Data Type	(Typical) Size in Bits
<code>bool</code>	1
<code>char</code>	8
<code>int</code>	32
<code>long</code>	64
<code>float</code>	32
<code>double</code>	64

Here, they are given with typical sizes in bits. In the C++ standard, sizes of fundamental data types are given as minimum required precision, not as a fixed size value. For example, the data type `int` requires at least 16 Bit / 2 Bytes, but usually comes with 32 Bit / 4 Bytes on modern computer architectures.

More on that here: <https://en.cppreference.com/w/cpp/language/types>.

This decision is rooted historically and is influenced by the need for portability and compatibility (e.g. backward compatibility) across various platforms, including those with different word sizes and memory constraints, e.g. in embedded systems.

C++ is a statically and strongly typed language, which means the types of variables must be known at compile-time.

This attribute enforces type safety by generating errors if variable types do not match during compilation, limiting implicit conversions.

The C++ cast operators are:

- `static_cast<T>`: Suitable for non-polymorphic type conversions, including upcasting and downcasting within class hierarchies, numeric type conversions, and pointer type conversions where implicit or reversible. This operator ensures type safety through compile-time checks.
- `dynamic_cast<T>`: Utilized for safe downcasting in class hierarchies, converting a base class pointer or reference to a derived class pointer or reference.
- `reinterpret_cast<T>`: Enables conversions between pointer types or between pointer and integer types. It can be used to "reinterpret" the memory, which can give a crucial speedup
- `const_cast<T>`: The only cast capable of modifying the constness of an object, useful for invoking non-const methods on const objects or modifying objects initially declared as const.

Type Casting (2)

// different static type-casting variants

```
short a = 10;
float b;
b = a;                // implicit (variant 0)
b = (float) a;        // explicit (variant 1)
b = float(a);         // explicit (variant 2)
b = static_cast<float>(a); // explicit (variant 3)
```

// dynamic type-casting for pointers/references to objects

```
Animal* animal = new Dog();
Dog* dog = dynamic_cast<Dog*>(animal);
```

// reinterpreting memory, here array interpreted as __m128 vector:

```
float data[] = {1.f, 2.f, 3.f, 4.f};
__m128 &vec = reinterpret_cast<__m128&>(data);
```

// modifying constness of an object to re-write const-objects:

```
int c = 10;
const int& d = c;           // create "modification-protected" reference
const_cast<int&>(d) = 5;    // modify "modification-protected" reference
```

A *variable* is an identifier that gives a name to an object (e.g. `int a;`).

A *literal* is a constant value of a certain type included in the source code (e.g. `a = 5;`).

An *expression* is a sequence of operators and their operands that specifies a computation (e.g. `a++;`).

Beside the data type and the object size, each object has two main properties

- An address - a memory location where it is stored
- Value/Content - data stored at the memory location of the variable

```
// create an integer variable at some free memory location, e.g. 0x7ffcdf8a990c  
// and set its value to 42.  
int a = 42;  
// only changing the value/content of a:  
a = 1337;
```

Initialization in C++

Traditional initialization in C++ has several forms:

- Direct Initialization: `int a(5);`
- Copy Initialization: `int a = 5;`

However, starting with C++ 11, *Brace Initialization* offers advantages:

- uniform syntax for all types.
- prevents narrowing conversions.
- can initialize aggregates, like arrays and structs, uniformly.

Narrowing Conversions:

```
int a{4};           // works fine
int b{7.9};         // compiler error: prevents lossy conversion
```

Works for primitives, aggregates, and classes:

```
std::vector<int> v{1, 2, 3};
int array[]{1, 2, 3};
YourClass obj{};
```

Using `{ }` reduces the likelihood of errors and enhances code clarity and safety, making it a recommended practice in modern C++.

Pointers (1)

In C++, it is quite common to work with pointers.

Pointers are variables that "point" to a specific memory location of an object. For using pointers, the following should be considered:

- Pointer objects always have the same size (e.g. 8 bytes). Moving or copying the pointer is often faster than the large data structure they are pointing to.
- Working on an object a pointer is pointing to changes the original object. The effect of changes is global.
- Pointers can be used to refactor data fast, e.g. swapping nodes in a graph tree structure does not require to move all child-nodes, when the structure works with pointers.

```
int a = 5;    // 5 stored at some memory location, let's say at 0x7ffcdf8a990c
int* b = &a;  // b points to the address of a (0x7ffcdf8a990c)
              // b itself is located somewhere else, e.g. at 0x7ffcdf8a9900
```


Pointers (2)

The value/content of a pointer is a memory address. The memory address where the pointer is pointing to has some content, which can be accessed by dereferencing the pointer.

```
std::cout << "address of a: " << &a << " is equal to the content of b: " << b << std::endl;
std::cout << "content of a: " << a << " is equal to the dereferenced b: " << *b << std::endl;
std::cout << "address of b: " << &b << std::endl;
```

Operations can be applied to pointers (e.g. move to another memory location by adding some value to the content of the pointer).

```
int arr[2];
int* p = arr; // same as "int* p = &arr[0]" -> p points to the first element
++p;          // p points now to the second element
```

In this example, the behavior of `arr` is similar to pointer: one can either set `p` to `arr` or to the address of element 0 of the array, but not to the address of `arr`.

It seems like the content of `arr` is an address, similar to pointers, but actually `arr` is a array-type. However, when used in expressions, the type is decaying into a pointer to its first element.

This behavior is a design choice in the C language (and inherited by C++) to keep the language simple and efficient.

Please also note: In C++, the keyword `this` which is used to refer to the object itself is also a pointer.

Excursion: The main Function's Type Decay

The main function was already shown with parameters:

```
int main(int argc, char** argv) { ... } // with function parameters argument count argc  
                                     // and argument vector argv
```

However, sometimes the main function in C++ is also defined as

```
int main(int argc, char* argv[]) { ... } // "old-school" with C-style array declaration
```

Although this old-fashioned definition might be more intuitive, it actually hides the *type decay* mentioned on the previous slide. As mentioned before, an array passed as a parameter is decaying into a pointer, which means that `char* argv[]` is decaying into `char** argv`, losing its array information.

Excursion: Preincrement and Postincrement

To move the pointer on the pre-previous slide, we incremented the pointer by using the overloaded operator++. C++ allows both, pre- and postincrement.

```
int a = 42;
std::cout << ++a << std::endl;    // prints 43
std::cout << a++ << std::endl;    // prints also 43, but a = 44 afterwards
std::cout << a << std::endl;      // prints 44
```

General rule: If you don't need post-increment explicitly, use pre-increment.

In many cases, for example in the condition expressions of loops to increment the counter, it does not really matter.

In several other cases it does:

- pre-increment is often more efficient because it doesn't need to create a temporary object (copy) to hold the original value before incrementing
- additional overhead can be unnecessary if you don't specifically need the original value before the increment
- some iterators for containers only have a defined pre-increment for performance reasons (consistency)

References in C++

References allow you to create an alias for an existing variable.

Working with a copy:

```
Matrix matrix = some_object.GetLargeMatrix(); // takes a lot of time
matrix.DoSomething();                          // works on the copy
```

Working with a reference:

```
Matrix& matrix = some_object.GetLargeMatrix(); // is very fast
matrix.DoSomething();                          // works on original
```

To use a reference, the return type of the function has to be changed to return a reference of the specific type!

A reference is implemented similarly to a pointer (e.g. it has a small size of 8 bytes in 64-Bit systems), while a copy is always depending on the target object's size.

When working with large data structures, copying a large object is timely expensive.

- 1. Memory has to be allocated for the new object
- 2. The whole object has to be copied into the new location

Avoiding unnecessary copies not only saves time but also reduces memory usage, which is critical in high-performance computing and memory-constrained environments.

Please note: You can not pass *literals* as reference!

The access to functions or variables of objects in C++ is depending on the type of the object.

When a pointer is involved, the $\rightarrow$ is used to access the objects' members:

```
Matrix* matrix = new Matrix();           // heap allocated Matrix object
matrix->DoSomething();                    // arrow is used, since matrix is a pointer object

// inside some class
this->size_ = 20;                         // 'this' works like a pointer to the object itself
```

For non-pointer objects, the dot is used to access object members:

```
Matrix matrix = Matrix();                 // stack allocated Matrix object
matrix.DoSomething();                     // dot is used, since matrix is not a pointer

Matrix &matrixRef = GetSomeMatrixRef();
matrixRef.DoSomething();                  // for a reference, the dot is used as well
```

Parameter-Passing Strategies: Pass by { Value, Reference, Pointer }

Please note: These strategies are also sometimes referred to as "call-by-{ Value, Reference, Pointer}". However, "pass-by" is becoming more commonly used and is often considered more precise in modern contexts.

Examples of functions with Pass-by-Value, Pass-by-Reference and Pass-by-Pointer.

Pass-by-Value:

```
int increment(int a)    // takes a copy of a
{
    return ++a;         // returns the incremented copy
}
```

Pass-by-Reference:

```
void increment(int& a)  // takes a reference to a
{
    ++a;                // increments the original a
}
```

Pass-by-Pointer:

```
void increment(int* p)  // takes a pointer to some integer
{
    ++(*p);             // dereferences the pointer and pre-increments afterwards
}
```

What is Const-Correctness?

- Ensuring that variables which should not change after initialization do not change
- It's about correctness and self-documenting code
- `const` can be applied to variables, pointers, methods, and more

Why use `const`?

- **Safety:** Prevents modification of data, which can lead to fewer bugs (e.g. when working with references)
- **Optimization:** Allows compiler optimizations on the assumption that values don't change
- **Clarity:** Makes the code easier to understand regarding what should not be altered

Example: Constness

```
// add constness as function parameter in the function signature
void do_something(const std::vector<int>& vector)
{
    vector.push_back(1); // error, modification of const object
}

const int max_value{42}; // constant integer

void print() const; // constant member function
```

Undefined Behavior (UB)

Undefined behavior occurs when code executes operations that have no specification by the C++ standard.

Literally anything can happen!

- It results from code that is syntactically correct but fails to adhere to certain rules or assumptions of the language.
- Examples include accessing uninitialized variables, out of bounds array access, null pointer dereferencing, and accessing deallocated memory.
- The compiler is not required to diagnose undefined behaviors. The program may compile but produce unpredictable results.
- Consequences can range from no apparent error, to crashes, or even security vulnerabilities.

The following example usually runs fine, but creates nonsense. Creating a pointer (here p) to a memory location that is freed afterwards, requires to manually handle the self-declared pointer. If not, accessing it is undefined behavior.

Bad example:

```
int* arr = new int[10];           // allocate memory
int* p = &arr[0];                 // set pointer to allocated memory
delete[] arr;                    // free allocated memory, pointer still exists
std::cout << *p << std::endl;    // output: 8042 --> UB!
```

Best Practice: Always initialize variables, check array bounds, avoid dereferencing null pointers, and adhere to the language's rules to prevent undefined behavior. In modern C++, also try to avoid raw pointers and use smart-pointers instead.

Stack- and Heap-Memory Allocation (1)

In C++, the virtual memory that can be allocated for an application is separated in a stack and a heap.

Stack-Memory:

- used for static memory allocation
- variables are automatically allocated and deallocated
- lifetime is tied to the containing block's execution (e.g., a function scope)
- fast allocation and deallocation due to its LIFO (Last In, First Out) nature
- limited size, can lead to stack overflow if excessively used

Heap-Memory:

- used for dynamic memory allocation
- variables must be manually allocated and deallocated (e.g., using `new` and `delete`)
- lifetime is controlled by the programmer, independent of scope
- allows for allocation of large amounts of memory, but at the cost of higher overhead
- improper management can lead to memory leaks and fragmentation

Understanding the differences in variable lifetime between the stack and heap is crucial for efficient memory management and avoiding common errors like leaks or overflow.

Arrays and other Containers

Arrays are known for fast access time by a given index, since its elements are stored in adjacent memory cells.

```
int arr[10];           // stack allocated C-style array
int* arr = new int[10]; // heap allocated C-style array
...
delete[] arr;          // mandatory: free heap allocated memory
```

Please note: If `arr` goes out of scope before calling the `delete` function to free the memory, the memory location is not accessible anymore, but still allocated → memory leak!

The size of an array allocated on the stack has to be known at compile time. Dynamic arrays on the other hand can be initialized at runtime, but require memory allocation on the heap.

```
int size;
std::cin >> size;           // expect user input at runtime
int arr[size];              // error, instead: int* arr = new int[size];
```

Additionally, an array allocated on the heap is accessible by a raw pointer that provides the data type and the memory position where the array starts. The size, however, has to be stored manually. Otherwise, one can easily run out of bounds and crash the application.

Template Class `std::vector` (1)

Raw heap allocation requires the programmer to take care about the lifetime of objects manually. To avoid memory leaks and include some quality-of-life features for arrays, there is a template class that can be used instead:

```
std::vector<T>
```

```
#include <vector>
std::vector<int> data(10);
```

In this example, the vector itself is stored at the stack. If the vector goes out of scope, memory is freed automatically. The class instance attributes, e.g. the dynamic array behind, is allocated on the heap and allows dynamic memory allocation.

Data can be accessed similarly to an array by using `vector[index]` (fast, without boundary check) or `vector.at(index)` (with boundary check, but slightly slower).

Template Class `std::vector` (2)

The `std::vector<T>` has a `push_back( ... )` function to add an element at the end of the vector. However, this can be timely expensive.

If the current vector is full, `push_back( ... )` automatically resizes the underlying array by $N/2$.

If the current memory location does not allow to resize, because it can not store $N/2$ new elements there, the whole vector has to be copied to a new location that can store $N + N/2$ elements. **Expensive!**

If the size is known, create the array with the correct size immediately or use the `reserve(n)` function to reserve enough memory space before working with it.

```
std::vector<int> data;
std::cout << data[0] << std::endl;    // UB, there is no element 0 yet.
std::cout << data.at(0) << std::endl; // Error, there is no element 0 yet.
data.reserve(1);
std::cout << data[0] << std::endl;    // UB, there is still no element 0 yet.
std::cout << data.at(0) << std::endl; // Error, there is still no element 0 yet.
data.push_back(42);
std::cout << data[0] << std::endl;    // output: 42
```

The `resize(n)` function can be used similarly, but also increases the actual size of the array (it adds value-initialized elements).

```
std::vector<int> data;
std::cout << data[0] << std::endl; // UB, there is no element 0 yet.
data.resize(1);
std::cout << data[0] << std::endl; // output: 0 (value-initialization for int)
```

The counterpart for `std::vector<T>` as a dynamically allocated container is `std::array<T, N>`.

Similarly to the stack-allocated raw C-style arrays, `std::array<T, N>` is allocated on the stack (size has to be known at compile-time!), but is usually preferred in modern C++.

```
int arr[10];                                // absolutely okay if required locally

#include <array>
std::array<int, 10> arr;                     // preferred when used more generalized
```

The implementation of `std::array<T, N>` allows to use many of the standard library's functions (e.g. for sorting), plus it is providing type safety and preventing type decay.

Type decay: In C++, a raw array can not be passed as a function argument or return statement. One can set a pointer to the array and pass the pointer, but then information (e.g. the size of the array) is lost.

The template classes `std::vector` and `std::array` were already shown without speaking about templates themselves.

These containers should be able to store data of a specific type. However, for all types, the behavior is the same. For instance, when storing numbers, accessing element 0 should give us the first element, no matter if it is an `int`, `float`, `double` or something else.

C++ allows to program templates, which can be classes, structs or functions where the programmer provides placeholders for types that are defined when the class or function is used.

```
template <typename T>
class MyTemplateClass {
...
}
```

In this class template example, `T` is the placeholder for a type. Inside the class, one can use `T` like any other type. `T` is replaced with the correct type when the class is used, e.g.:

```
MyArrayClass arr = MyArrayClass<int>(...)
```

In this instance, all `T` are replaced with `int`.

Using templates helps to avoid multiple definitions for each type and, therefore, to reduce the amount of code that has to be maintained. The programmer has to make sure that templates work as expected for a given type.